Lucid — Complete Backend API Contract Specification

Document Version: 2026.11-v2.0
Authoritative Date: 2026-11-08
Target Audience: Frontend Engineers, Backend Developers, QA Automation Engineers, Integration Architects
Scope: Complete field-level contract specification for every API endpoint implemented in the Lucid Indian Equity Portfolio Management Platform.

Executive Summary & Design Scope

This document provides the authoritative, exhaustive API contract for **Lucid**. It specifies all **82 unique API paths** representing **88 HTTP operations** registered in the FastAPI application.

Every endpoint specification documents:

1. **API Overview & Identity:** Exact route, HTTP method, source router, and OpenAPI operation ID.
2. **Business Purpose & Context:** Why the endpoint exists and what specific problem it solves in Indian equity portfolio management.
3. **Execution Trigger:** Which React 19 frontend component or backend daemon calls it and when.
4. **Authentication & Authorization:** Cryptographic Bearer JWT requirements, Argon2id security, and tenant isolation guardrails.
5. **Request Specification:** Path parameters, query parameters, request headers, and complete Pydantic request bodies with field types, validation constraints, and representative JSON payloads.
6. **Internal Processing & Service Lineage:** End-to-end data processing flow through routers, services, LangGraph agents, SLSQP optimizers, and PostgreSQL transactions.
7. **Response Specification:** HTTP status codes, response schemas, field-level dictionary, and representative response JSON payloads.
8. **Error Handling Matrix:** Exhaustive breakdown of HTTP error responses (400, 401, 403, 404, 422, 429, 500) with error conditions, causes, response bodies, and resolutions.
9. **Database Lineage:** Exact tables read, inserted, updated, and deleted.
10. **External Provider Dependencies:** NSE, BSE, Yahoo Finance (`yfinance`), Google Gemini (`gemini-3.8-flash`), and RBI feeds.
11. **Executable cURL Example:** Ready-to-run cURL command with authentic headers and expected responses.
12. **Related API Workflow:** Preceding and succeeding endpoints in the user journey.

Table of Contents

1. API Design Standards & Architectural Conventions
2. Complete Master API Inventory
3. Domain 1: Authentication & Account Management (13 Endpoints)
4. Domain 2: Financial Goals Management (4 Endpoints)
5. Domain 3: Investor Profile & Preferences (7 Endpoints)
6. Domain 4: Risk Assessment & Questionnaire (3 Endpoints)
7. Domain 5: Portfolio Formulation, Optimization & Explainability (25 Endpoints)
8. Domain 6: Demat Portfolio Diagnosis & AI Copilot (9 Endpoints)
9. Domain 7: Market Data, Security Master & Regime Ingestion (13 Endpoints)
10. Domain 8: Notifications & System Alerts (5 Endpoints)
11. Domain 9: Link Integrity, Disclosures & Regulatory Verifications (4 Endpoints)
12. Domain 10: System Health, Database Statistics & Diagnostics (5 Endpoints)
13. Frontend-to-Backend API Mapping (All 24 Components)
14. API Workflow & Sequence Diagrams
15. Postman Suite & Verification Consistency

---

1. API Design Standards & Architectural Conventions

1.1 Base URLs and Versioning

- **Base URL:** `http://localhost:8000` (Development) / `https://api.lucid.investments` (Production).
- **API Versioning:** All operational endpoints reside under `/api/v1/`. Infrastructure and health checks reside at `/api/health` and `/health`.

1.2 Authentication & Security Headers

- **Authentication Scheme:** `Authorization: Bearer <JWT_TOKEN>`.
- **Password Hashing:** Canonical **Argon2id** (`argon2-cffi>=25.1.0`) with time cost = 3, memory cost = 65,536 KB, parallelism = 4.
- **Token Invalidation:** Revoked tokens are persisted to the PostgreSQL `revoked_tokens` table.
- **Security Headers:** Every response enforces `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and `Permissions-Policy`.

1.3 Data Types and Indian Market Formats

- **Currency:** Indian Rupee (`INR`), formatted as floating point (e.g. `105000.50`).
- **Dates & Timestamps:** ISO 8601 extended format with UTC offset (e.g. `2026-11-08T09:15:00+05:30`).
- **Percentages:** Represented as percentages from 0.0 to 100.0 in presentation contracts and 0.0 to 1.0 in optimizer math.

---

2. Complete Master API Inventory

Refer to API_INVENTORY.md for the searchable master table of all 88 HTTP operations.

---

1. Authentication and Account Management

`POST /api/v1/auth/login` — Authenticate user and issue JWT token pair

A. API Overview:

- **Endpoint:** `/api/v1/auth/login`
- **HTTP Method:** `POST`
- **Summary:** Authenticate user and issue JWT token pair
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade authenticate user and issue jwt token pair. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "email": "investor@lucid.local",
  "password": "SecurePassword123!"
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/login)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsIn...",
  "token_type": "bearer",
  "expires_in": 86400,
```

```
  "user_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/login" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/token` — Issue OAuth2-compatible bearer token

A. API Overview:

- **Endpoint:** `/api/v1/auth/token`
- **HTTP Method:** `POST`
- **Summary:** Issue OAuth2-compatible bearer token
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `api.ts`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade issue oauth2-compatible bearer token. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `api.ts` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "email": "investor@lucid.local",
  "password": "SecurePassword123!"
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/token)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
```

```
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsIn...",
  "token_type": "bearer",
  "expires_in": 86400,
  "user_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/token" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/register` — Register a new investor account with Argon2id hashing

A. API Overview:

- **Endpoint:** `/api/v1/auth/register`
- **HTTP Method:** `POST`
- **Summary:** Register a new investor account with Argon2id hashing
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade register a new investor account with argon2id hashing. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.

- **Request Body Content-Type:** `application/json`

```
{
  "email": "investor@lucid.local",
  "password": "SecurePassword123!",
  "display_name": "Ankur Sharma"
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/register)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `201 Created`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/register" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/request-verification` — Request email or SMS verification OTP

A. API Overview:

- **Endpoint:** `/api/v1/auth/request-verification`
- **HTTP Method:** `POST`
- **Summary:** Request email or SMS verification OTP
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`

- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade request email or sms verification otp. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/request-verification)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/request-verification" \
```

```
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/verify-code` — Verify OTP security code

A. API Overview:

- **Endpoint:** `/api/v1/auth/verify-code`
- **HTTP Method:** `POST`
- **Summary:** Verify OTP security code
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade verify otp security code. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/verify-code)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided

HTTP Status	Error Condition	Cause	Resolution
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/verify-code" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/forgot-password` — Initiate password recovery with temporary token

A. API Overview:

- **Endpoint:** `/api/v1/auth/forgot-password`
- **HTTP Method:** `POST`
- **Summary:** Initiate password recovery with temporary token
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade initiate password recovery with temporary token. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/forgot-password)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/forgot-password" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/reset-password` — Reset password using verified recovery token

A. API Overview:

- **Endpoint:** `/api/v1/auth/reset-password`
- **HTTP Method:** `POST`
- **Summary:** Reset password using verified recovery token
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `AuthView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade reset password using verified recovery token. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `AuthView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/reset-password)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
```

```
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/reset-password" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/change-password` — Change password for authenticated user

A. API Overview:

- **Endpoint:** `/api/v1/auth/change-password`
- **HTTP Method:** `POST`
- **Summary:** Change password for authenticated user
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade change password for authenticated user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/change-password)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/change-password" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/logout` — Revoke active JWT token and persist to blacklist

A. API Overview:

- **Endpoint:** `/api/v1/auth/logout`
- **HTTP Method:** `POST`
- **Summary:** Revoke active JWT token and persist to blacklist
- **Router Module:** `backend/api/v1/auth.py`

- **Frontend Caller:** `Navbar.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade revoke active jwt token and persist to blacklist. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `Navbar.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/logout)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/logout" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`GET /api/v1/auth/me` — Retrieve current authenticated user profile snapshot

A. API Overview:

- **Endpoint:** `/api/v1/auth/me`
- **HTTP Method:** `GET`
- **Summary:** Retrieve current authenticated user profile snapshot
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `Navbar.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve current authenticated user profile snapshot. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `Navbar.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/auth/me)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership

HTTP Status	Error Condition	Cause	Resolution
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/auth/me" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` ➔ `GET /api/v1/auth/me` ➔ `GET /api/v1/profile`

`GET /api/v1/auth/export-data` — Export complete user data archive (GDPR/DPDP)

A. API Overview:

- **Endpoint:** `/api/v1/auth/export-data`
- **HTTP Method:** `GET`
- **Summary:** Export complete user data archive (GDPR/DPDP)
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade export complete user data archive (gdpr/dpdp). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/auth/export-data)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/auth/export-data" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` ➔ `GET /api/v1/auth/me` ➔ `GET /api/v1/profile`

`DELETE /api/v1/auth/account` — Permanently delete account (Legacy route)

A. API Overview:

- **Endpoint:** `/api/v1/auth/account`
- **HTTP Method:** `DELETE`
- **Summary:** Permanently delete account (Legacy route)
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade permanently delete account (legacy route). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/auth/account)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
```

```
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/auth/account" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

`POST /api/v1/auth/delete-account` — Permanently delete account with password verification

A. API Overview:

- **Endpoint:** `/api/v1/auth/delete-account`
- **HTTP Method:** `POST`
- **Summary:** Permanently delete account with password verification
- **Router Module:** `backend/api/v1/auth.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade permanently delete account with password verification. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
```

```
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/auth/delete-account)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `app_users`, `user_profiles`, `revoked_tokens`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/auth/delete-account" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/auth/login` → `GET /api/v1/auth/me` → `GET /api/v1/profile`

---

2. Financial Goals Management

`GET /api/v1/goals` — List all active financial goals for current user

A. API Overview:

- **Endpoint:** `/api/v1/goals`
- **HTTP Method:** `GET`
- **Summary:** List all active financial goals for current user
- **Router Module:** `backend/api/v1/goals.py`
- **Frontend Caller:** `OnboardingWizard.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list all active financial goals for current user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `OnboardingWizard.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/goals)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `financial_goals`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/goals" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/goals` — Create a new financial investment goal

A. API Overview:

- **Endpoint:** `/api/v1/goals`
- **HTTP Method:** `POST`
- **Summary:** Create a new financial investment goal
- **Router Module:** `backend/api/v1/goals.py`
- **Frontend Caller:** `OnboardingWizard.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade create a new financial investment goal. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `OnboardingWizard.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "goal_name": "Retirement Corpus",
  "target_amount": 25000000.0,
  "target_horizon_months": 120,
  "priority_level": "HIGH"
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/goals)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `201 Created`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `financial_goals`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/goals" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`PATCH /api/v1/goals/{goal_id}` — Update financial goal target amount or horizon

A. API Overview:

- **Endpoint:** `/api/v1/goals/{goal_id}`
- **HTTP Method:** `PATCH`
- **Summary:** Update financial goal target amount or horizon
- **Router Module:** `backend/api/v1/goals.py`
- **Frontend Caller:** `OnboardingWizard.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade update financial goal target amount or horizon. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `OnboardingWizard.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `goal_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "goal_name": "Retirement Corpus",
  "target_amount": 25000000.0,
  "target_horizon_months": 120,
  "priority_level": "HIGH"
}
```

F. Processing and Internal Workflow:

```
Client Request (PATCH /api/v1/goals/{goal_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`

- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `financial_goals`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X PATCH "http://localhost:8000/api/v1/goals/g1" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`DELETE /api/v1/goals/{goal_id}` — Delete an existing financial goal

A. API Overview:

- **Endpoint:** `/api/v1/goals/{goal_id}`
- **HTTP Method:** `DELETE`
- **Summary:** Delete an existing financial goal
- **Router Module:** `backend/api/v1/goals.py`
- **Frontend Caller:** `OnboardingWizard.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade delete an existing financial goal. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `OnboardingWizard.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `goal_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/goals/{goal_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
```

```
      ▼
JWT Verification (Security Bearer Dependency)
      │
      ▼
Pydantic Input Schema Validation
      │
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `financial_goals`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/goals/g1" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

3. Investor Profile & Preferences

`GET /api/v1/profile` — Retrieve investor profile and investment experience

A. API Overview:

- **Endpoint:** `/api/v1/profile`
- **HTTP Method:** `GET`
- **Summary:** Retrieve investor profile and investment experience
- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve investor profile and investment experience. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/profile)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/profile" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`DELETE /api/v1/profile` — Delete user profile and account

A. API Overview:

- **Endpoint:** `/api/v1/profile`
- **HTTP Method:** `DELETE`
- **Summary:** Delete user profile and account

- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade delete user profile and account. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/profile)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/profile" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
```

```
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`PATCH /api/v1/profile` — Update investor profile details and currency

A. API Overview:

- **Endpoint:** `/api/v1/profile`
- **HTTP Method:** `PATCH`
- **Summary:** Update investor profile details and currency
- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade update investor profile details and currency. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (PATCH /api/v1/profile)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership

HTTP Status	Error Condition	Cause	Resolution
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X PATCH "http://localhost:8000/api/v1/profile" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/profile/versions` — Retrieve historical audit versions of investor profile

A. API Overview:

- **Endpoint:** `/api/v1/profile/versions`
- **HTTP Method:** `GET`
- **Summary:** Retrieve historical audit versions of investor profile
- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve historical audit versions of investor profile. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/profile/versions)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/profile/versions" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` ➔ `GET /api/v1/market-data/health`

`POST /api/v1/profile/extract-from-text` — Extract structured profile from natural language prompt

A. API Overview:

- **Endpoint:** `/api/v1/profile/extract-from-text`
- **HTTP Method:** `POST`
- **Summary:** Extract structured profile from natural language prompt
- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade extract structured profile from natural language prompt. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/profile/extract-from-text)
      |
```

```
      ▼
FastAPI Security Header & Rate Limiter Middleware
      │
      ▼
JWT Verification (Security Bearer Dependency)
      │
      ▼
Pydantic Input Schema Validation
      │
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/profile/extract-from-text" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/profile/export` — Export comprehensive investor data package

A. API Overview:

- **Endpoint:** `/api/v1/profile/export`
- **HTTP Method:** `GET`
- **Summary:** Export comprehensive investor data package
- **Router Module:** `backend/api/v1/profile.py`
- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade export comprehensive investor data package. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)

- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/profile/export)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/profile/export" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/profile/delete-account` — Permanent account deletion with confirmation phrase

A. API Overview:

- **Endpoint:** `/api/v1/profile/delete-account`
- **HTTP Method:** `POST`
- **Summary:** Permanent account deletion with confirmation phrase
- **Router Module:** `backend/api/v1/profile.py`

- **Frontend Caller:** `ProfileAuthModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade permanent account deletion with confirmation phrase. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ProfileAuthModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/profile/delete-account)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/profile/delete-account" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

4. Risk Assessment & Questionnaire

`GET /api/v1/risk-questions` — Retrieve 8-question psychometric risk questionnaire

A. API Overview:

- **Endpoint:** `/api/v1/risk-questions`
- **HTTP Method:** `GET`
- **Summary:** Retrieve 8-question psychometric risk questionnaire
- **Router Module:** `backend/api/v1/risk-questions.py`
- **Frontend Caller:** `RiskQuestionnaireView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve 8-question psychometric risk questionnaire. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `RiskQuestionnaireView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/risk-questions)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership

HTTP Status	Error Condition	Cause	Resolution
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/risk-questions" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/risk-assessments` — Submit questionnaire answers and compute risk score

A. API Overview:

- **Endpoint:** `/api/v1/risk-assessments`
- **HTTP Method:** `POST`
- **Summary:** Submit questionnaire answers and compute risk score
- **Router Module:** `backend/api/v1/risk-assessments.py`
- **Frontend Caller:** `RiskQuestionnaireView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade submit questionnaire answers and compute risk score. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `RiskQuestionnaireView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "answers": [{"question_id": "q1", "option_id": "o2"}],
  "investment_experience_years": 5
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/risk-assessments)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/risk-assessments" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/risk-assessments/latest` — Retrieve latest completed risk assessment score

A. API Overview:

- **Endpoint:** `/api/v1/risk-assessments/latest`
- **HTTP Method:** `GET`
- **Summary:** Retrieve latest completed risk assessment score
- **Router Module:** `backend/api/v1/risk-assessments.py`
- **Frontend Caller:** `RiskQuestionnaireView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve latest completed risk assessment score. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `RiskQuestionnaireView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/risk-assessments/latest)
      |
      ▼
```

```
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/risk-assessments/latest" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

5. Portfolio Generation & Version Management

`POST /api/v1/portfolio-generation-requests` — Initiate multi-agent portfolio generation workflow

A. API Overview:

- **Endpoint:** `/api/v1/portfolio-generation-requests`
- **HTTP Method:** `POST`
- **Summary:** Initiate multi-agent portfolio generation workflow
- **Router Module:** `backend/api/v1/portfolio-generation-requests.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade initiate multi-agent portfolio generation workflow. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "investment_amount": 500000.0,
  "currency_code": "INR",
  "investment_horizon_months": 36,
  "risk_category": "BALANCED",
  "goal_type": "WEALTH_CREATION",
  "user_prompt": "Focus on high-quality large-cap and growth mid-caps."
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolio-generation-requests)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `201 Created`
- **Response Content-Type:** `application/json`

```
{
  "request_id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
  "status": "COMPLETED",
  "portfolio_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "total_invested": 498420.0,
  "unallocated_cash": 1580.0
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolio-generation-requests" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/portfolio-generation-requests/{request_id}` — Retrieve status and telemetry of generation request

A. API Overview:

- **Endpoint:** `/api/v1/portfolio-generation-requests/{request_id}`
- **HTTP Method:** `GET`
- **Summary:** Retrieve status and telemetry of generation request
- **Router Module:** `backend/api/v1/portfolio-generation-requests.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve status and telemetry of generation request. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:**
  - `request_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolio-generation-requests/{request_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "request_id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
  "status": "COMPLETED",
  "portfolio_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "total_invested": 498420.0,
  "unallocated_cash": 1580.0
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolio-generation-requests/{request_id}" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/portfolios` — List all portfolios belonging to current user

A. API Overview:

- **Endpoint:** `/api/v1/portfolios`
- **HTTP Method:** `GET`
- **Summary:** List all portfolios belonging to current user
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list all portfolios belonging to current user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`

HTTP Status	Error Condition	Cause	Resolution
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`DELETE /api/v1/portfolios` — Delete all portfolios for current user

A. API Overview:

- **Endpoint:** `/api/v1/portfolios`
- **HTTP Method:** `DELETE`
- **Summary:** Delete all portfolios for current user
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade delete all portfolios for current user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/portfolios)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/portfolios" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`POST /api/v1/portfolios/batch-delete` — Batch delete selected portfolios

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/batch-delete`
- **HTTP Method:** `POST`
- **Summary:** Batch delete selected portfolios
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade batch delete selected portfolios. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/batch-delete)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/batch-delete" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`DELETE /api/v1/portfolios/{portfolio_id}` — Permanently delete an individual portfolio

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}`
- **HTTP Method:** `DELETE`
- **Summary:** Permanently delete an individual portfolio
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `DeletePortfolioModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade permanently delete an individual portfolio. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DeletePortfolioModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/portfolios/{portfolio_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/versions` — List historical versions for a portfolio

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions`
- **HTTP Method:** `GET`
- **Summary:** List historical versions for a portfolio
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `VersionComparisonModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list historical versions for a portfolio. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `VersionComparisonModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/versions)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}` — Retrieve details of a specific portfolio version

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions/{version_id}`
- **HTTP Method:** `GET`
- **Summary:** Retrieve details of a specific portfolio version
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve details of a specific portfolio version. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
  - `version_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/versions/{version_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions/c9a0b1c2-d3e4-5f6a-7b8c-9d0e1f2a3b4c
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/explanations` — Retrieve multi-agent research explanations

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions/{version_id}/explanations`
- **HTTP Method:** `GET`
- **Summary:** Retrieve multi-agent research explanations
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `ExplainabilityView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve multi-agent research explanations. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ExplainabilityView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
  - `version_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/explanations)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
```

```
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions/c9a0b1c2-d3e4-5f6a-7b8c-9d0e1f2a3b4c
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/why` — Retrieve dual-level stock justification rationale

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/why`
- **HTTP Method:** `GET`
- **Summary:** Retrieve dual-level stock justification rationale
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `StockWhyModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve dual-level stock justification rationale. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `StockWhyModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
  - `version_id` (UUID/String, required): Target entity identifier.

    - `symbol` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/why)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "symbol": "TCS",
  "role": "CORE",
  "summary": "Selected for industry-leading ROCE (>50%) and zero debt.",
  "fundamental_evidence": {"roce": 52.4, "debt_to_equity": 0.0}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions/c9a0b1c2-d3e4-5f6a-7b8c-9d0e1f2a3b4c
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/chat` — Interactive chat about specific stock holding

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/chat`
- **HTTP Method:** `POST`
- **Summary:** Interactive chat about specific stock holding
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `StockWhyModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade interactive chat about specific stock holding. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `StockWhyModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
  - `version_id` (UUID/String, required): Target entity identifier.
  - `symbol` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/versions/{version_id}/stocks/{symbol}/chat)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions/c9a0b1c2-d3e4-5f6a-7b8c-9d0e1f2a3b4
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`POST /api/v1/portfolios/compare` — Universal side-by-side comparison of portfolios

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/compare`
- **HTTP Method:** `POST`
- **Summary:** Universal side-by-side comparison of portfolios
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `VersionComparisonModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade universal side-by-side comparison of portfolios. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `VersionComparisonModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/compare)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/compare" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/compare` — Compare two versions of the same portfolio

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/compare`
- **HTTP Method:** `POST`
- **Summary:** Compare two versions of the same portfolio
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `VersionComparisonModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade compare two versions of the same portfolio. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `VersionComparisonModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/compare)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
```

```
      ▼
Pydantic Input Schema Validation
      │
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/compare" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/scenarios` — Execute macroeconomic stress-testing scenarios

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/scenarios`
- **HTTP Method:** `POST`
- **Summary:** Execute macroeconomic stress-testing scenarios
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `StressScenariosView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade execute macroeconomic stress-testing scenarios. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `StressScenariosView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/scenarios)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/scenarios" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/benchmark-evaluation` — Evaluate portfolio performance against Nifty 50/500

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/benchmark-evaluation`
- **HTTP Method:** `GET`

- **Summary:** Evaluate portfolio performance against Nifty 50/500
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `BenchmarksView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade evaluate portfolio performance against nifty 50/500. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `BenchmarksView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/benchmark-evaluation)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/benchmark-evaluation" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/export-orders` — Export proposed orders as JSON or downloadable CSV

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/versions/{version_id}/export-orders`
- **HTTP Method:** `GET`
- **Summary:** Export proposed orders as JSON or downloadable CSV
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade export proposed orders as json or downloadable csv. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
  - `version_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/versions/{version_id}/export-orders)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`

HTTP Status	Error Condition	Cause	Resolution
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/versions/c9a0b1c2-d3e4-5f6a-7b8c-9d0e1f2a3b4c
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/monitoring` — Retrieve live valuation, NAV, and drift radar metrics

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/monitoring`
- **HTTP Method:** `GET`
- **Summary:** Retrieve live valuation, NAV, and drift radar metrics
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve live valuation, nav, and drift radar metrics. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/monitoring)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/monitoring" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/monitor-now` — Trigger immediate real-time portfolio valuation check

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/monitor-now`
- **HTTP Method:** `POST`
- **Summary:** Trigger immediate real-time portfolio valuation check
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade trigger immediate real-time portfolio valuation check. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/monitor-now)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/monitor-now" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/{portfolio_id}/rebalance-preview` — Generate proposed rebalance trade orders preview

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/rebalance-preview`
- **HTTP Method:** `GET`
- **Summary:** Generate proposed rebalance trade orders preview
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `RebalancePreviewModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade generate proposed rebalance trade orders preview. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `RebalancePreviewModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/{portfolio_id}/rebalance-preview)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/rebalance-preview" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/check-drift-and-trigger` — Evaluate drift thresholds and trigger rebalancing

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/check-drift-and-trigger`
- **HTTP Method:** `POST`
- **Summary:** Evaluate drift thresholds and trigger rebalancing
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade evaluate drift thresholds and trigger rebalancing. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/check-drift-and-trigger)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership

HTTP Status	Error Condition	Cause	Resolution
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/check-drift-and-trigger" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/apply-rebalance` — Confirm and apply rebalanced portfolio version

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/apply-rebalance`
- **HTTP Method:** `POST`
- **Summary:** Confirm and apply rebalanced portfolio version
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `RebalancePreviewModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade confirm and apply rebalanced portfolio version. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `RebalancePreviewModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "notes": "Approved quarterly portfolio rebalance schedule."
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/apply-rebalance)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
```

```
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/apply-rebalance" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`POST /api/v1/portfolios/{portfolio_id}/chat` — Interactive portfolio assistant chat

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/{portfolio_id}/chat`
- **HTTP Method:** `POST`
- **Summary:** Interactive portfolio assistant chat
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `ConversationalAssistant.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade interactive portfolio assistant chat. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ConversationalAssistant.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/{portfolio_id}/chat)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/f47ac10b-58cc-4372-a567-0e02b2c3d479/chat" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/chat` — General conversational assistant chat

A. API Overview:

- **Endpoint:** `/api/v1/chat`
- **HTTP Method:** `POST`
- **Summary:** General conversational assistant chat
- **Router Module:** `backend/api/v1/chat.py`
- **Frontend Caller:** `ConversationalAssistant.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade general conversational assistant chat. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ConversationalAssistant.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/chat)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/chat" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/portfolios/monitoring/run-scheduler-cycle` — Manually trigger background monitoring scheduler run

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/monitoring/run-scheduler-cycle`
- **HTTP Method:** `POST`
- **Summary:** Manually trigger background monitoring scheduler run
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade manually trigger background monitoring scheduler run. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/portfolios/monitoring/run-scheduler-cycle)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID

HTTP Status	Error Condition	Cause	Resolution
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/portfolios/monitoring/run-scheduler-cycle" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`GET /api/v1/portfolios/monitoring/scheduler-status` — Retrieve background monitoring scheduler telemetry

A. API Overview:

- **Endpoint:** `/api/v1/portfolios/monitoring/scheduler-status`
- **HTTP Method:** `GET`
- **Summary:** Retrieve background monitoring scheduler telemetry
- **Router Module:** `backend/api/v1/portfolios.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve background monitoring scheduler telemetry. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/portfolios/monitoring/scheduler-status)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`

- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/portfolios/monitoring/scheduler-status" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

---

6. Existing Portfolio Diagnosis

`GET /api/v1/diagnosis/csv-template` — Download sample CSV template for Demat holdings import

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/csv-template`
- **HTTP Method:** `GET`
- **Summary:** Download sample CSV template for Demat holdings import
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade download sample csv template for demat holdings import. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/diagnosis/csv-template)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/diagnosis/csv-template" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

`POST /api/v1/diagnosis/upload-csv` — Upload Demat statement CSV and import holdings

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/upload-csv`
- **HTTP Method:** `POST`
- **Summary:** Upload Demat statement CSV and import holdings
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade upload demat statement csv and import holdings. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "portfolio_name": "Zerodha Demat Holdings",
  "holdings": [{"symbol": "INFY", "shares": 50, "average_price": 1820.0}]
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/diagnosis/upload-csv)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/diagnosis/upload-csv" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

`POST /api/v1/diagnosis/parse-text` — Parse raw unstructured holdings text into structured records

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/parse-text`
- **HTTP Method:** `POST`
- **Summary:** Parse raw unstructured holdings text into structured records
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade parse raw unstructured holdings text into structured records. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/diagnosis/parse-text)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/diagnosis/parse-text" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

`GET /api/v1/diagnosis/portfolios` — List all imported external Demat portfolios

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/portfolios`
- **HTTP Method:** `GET`
- **Summary:** List all imported external Demat portfolios
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list all imported external demat portfolios. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/diagnosis/portfolios)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/diagnosis/portfolios" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/diagnosis/portfolios` — Create empty Demat portfolio container

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/portfolios`
- **HTTP Method:** `POST`
- **Summary:** Create empty Demat portfolio container
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade create empty demat portfolio container. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/diagnosis/portfolios)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
```

```
      │
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/diagnosis/portfolios" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` → `GET /api/v1/portfolios/{id}` → `GET .../monitoring` → `GET .../rebalance-preview`

`DELETE /api/v1/diagnosis/portfolios/{existing_portfolio_id}` — Delete an imported Demat portfolio

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/portfolios/{existing_portfolio_id}`
- **HTTP Method:** `DELETE`
- **Summary:** Delete an imported Demat portfolio
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade delete an imported demat portfolio. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `existing_portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/diagnosis/portfolios/{existing_portfolio_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `investor_portfolios`, `portfolio_versions`, `portfolio_security_decisions`, `recommendation_explanations`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/diagnosis/portfolios/e1" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/portfolio-generation-requests` ➔ `GET /api/v1/portfolios/{id}` ➔ `GET .../monitoring` ➔ `GET .../rebalance-preview`

`POST /api/v1/diagnosis/analyze/{existing_portfolio_id}` — Execute HHI concentration and diversification diagnosis

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/analyze/{existing_portfolio_id}`
- **HTTP Method:** `POST`
- **Summary:** Execute HHI concentration and diversification diagnosis
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade execute hhi concentration and diversification diagnosis. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `existing_portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/diagnosis/analyze/{existing_portfolio_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/diagnosis/analyze/e1" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

`GET /api/v1/diagnosis/{existing_portfolio_id}/explanation` — Retrieve detailed explanation of diagnosis results

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/{existing_portfolio_id}/explanation`
- **HTTP Method:** `GET`
- **Summary:** Retrieve detailed explanation of diagnosis results
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve detailed explanation of diagnosis results. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `existing_portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/diagnosis/{existing_portfolio_id}/explanation)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided

HTTP Status	Error Condition	Cause	Resolution
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/diagnosis/e1/explanation" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

`POST /api/v1/diagnosis/{existing_portfolio_id}/chat` — Interactive AI copilot chat for Demat diagnosis

A. API Overview:

- **Endpoint:** `/api/v1/diagnosis/{existing_portfolio_id}/chat`
- **HTTP Method:** `POST`
- **Summary:** Interactive AI copilot chat for Demat diagnosis
- **Router Module:** `backend/api/v1/diagnosis.py`
- **Frontend Caller:** `DiagnosisCopilotModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade interactive ai copilot chat for demat diagnosis. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisCopilotModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `existing_portfolio_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/diagnosis/{existing_portfolio_id}/chat)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `existing_portfolios`, `existing_portfolio_holdings`, `portfolio_diagnoses`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Google Gemini API (`gemini-3.8-flash`) with deterministic rule fallback.

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/diagnosis/e1/chat" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `POST /api/v1/diagnosis/upload-csv` → `POST /api/v1/diagnosis/analyze/{id}` → `POST /api/v1/diagnosis/{id}/chat`

---

7. Market Data & Universe Management

`GET /api/v1/market-data/health` — Check health and connectivity of market data feeds

A. API Overview:

- **Endpoint:** `/api/v1/market-data/health`
- **HTTP Method:** `GET`
- **Summary:** Check health and connectivity of market data feeds
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `HoldingsView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade check health and connectivity of market data feeds. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/health)
      |
      ▼
```

```
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/health" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/market-data/ingest` — Trigger manual market data price ingestion

A. API Overview:

- **Endpoint:** `/api/v1/market-data/ingest`
- **HTTP Method:** `POST`
- **Summary:** Trigger manual market data price ingestion
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade trigger manual market data price ingestion. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/market-data/ingest)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/market-data/ingest" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/ingestion-runs` — List historical market data ingestion runs

A. API Overview:

- **Endpoint:** `/api/v1/market-data/ingestion-runs`
- **HTTP Method:** `GET`
- **Summary:** List historical market data ingestion runs
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `HoldingsView.tsx`

- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list historical market data ingestion runs. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `HoldingsView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/ingestion-runs)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/ingestion-runs" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/prices/{symbol}` — Retrieve historical OHLCV daily price bars for a symbol

A. API Overview:

- **Endpoint:** `/api/v1/market-data/prices/{symbol}`
- **HTTP Method:** `GET`
- **Summary:** Retrieve historical OHLCV daily price bars for a symbol
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `StockWhyModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve historical ohlcv daily price bars for a symbol. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `StockWhyModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:**
  - `symbol` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/prices/{symbol})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Yahoo Finance API (`yfinance`), cached in PostgreSQL.

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/prices/TCS" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/regime` — Retrieve current market regime classification and VIX

A. API Overview:

- **Endpoint:** `/api/v1/market-data/regime`
- **HTTP Method:** `GET`
- **Summary:** Retrieve current market regime classification and VIX
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve current market regime classification and vix. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/regime)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "market_regime": "BULL_TREND",
  "india_vix": 14.2,
  "nifty_50_trend": "ABOVE_200_DMA"
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`

HTTP Status	Error Condition	Cause	Resolution
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: Yahoo Finance API (`yfinance`), cached in PostgreSQL.

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/regime" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/macro-indicators` — Retrieve Indian macroeconomic indicators (RBI, CPI)

A. API Overview:

- **Endpoint:** `/api/v1/market-data/macro-indicators`
- **HTTP Method:** `GET`
- **Summary:** Retrieve Indian macroeconomic indicators (RBI, CPI)
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `ExplainabilityView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve indian macroeconomic indicators (rbi, cpi). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ExplainabilityView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/macro-indicators)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/macro-indicators" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/news` — Retrieve recent BSE/NSE market news and disclosures

A. API Overview:

- **Endpoint:** `/api/v1/market-data/news`
- **HTTP Method:** `GET`
- **Summary:** Retrieve recent BSE/NSE market news and disclosures
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `ExplainabilityView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve recent bse/nse market news and disclosures. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `ExplainabilityView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/news)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
```

```
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/news" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/universe-stats` — Retrieve counts of stored and recommendation securities

A. API Overview:

- **Endpoint:** `/api/v1/market-data/universe-stats`
- **HTTP Method:** `GET`
- **Summary:** Retrieve counts of stored and recommendation securities
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve counts of stored and recommendation securities. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/universe-stats)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/universe-stats" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/market-data/sync-universe` — Trigger full NSE EQUITY_L.csv universe sync (2,639 stocks)

A. API Overview:

- **Endpoint:** `/api/v1/market-data/sync-universe`
- **HTTP Method:** `POST`
- **Summary:** Trigger full NSE EQUITY_L.csv universe sync (2,639 stocks)
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade trigger full nse equity_l.csv universe sync (2,639 stocks). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/market-data/sync-universe)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- Provider: NSE India Official Bulk Master (`EQUITY_L.csv`).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/market-data/sync-universe" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/reconcile-universe` — Reconcile exchange universe and candidate eligibility

A. API Overview:

- **Endpoint:** `/api/v1/market-data/reconcile-universe`
- **HTTP Method:** `GET`
- **Summary:** Reconcile exchange universe and candidate eligibility
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade reconcile exchange universe and candidate eligibility. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/reconcile-universe)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/reconcile-universe" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/market-data/securities/search` — Fuzzy search securities by name, symbol, or ISIN

A. API Overview:

- **Endpoint:** `/api/v1/market-data/securities/search`
- **HTTP Method:** `GET`
- **Summary:** Fuzzy search securities by name, symbol, or ISIN
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade fuzzy search securities by name, symbol, or isin. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/market-data/securities/search)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/market-data/securities/search" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/market-data/securities/resolve-text` — Resolve company names to canonical ISINs and symbols

A. API Overview:

- **Endpoint:** `/api/v1/market-data/securities/resolve-text`
- **HTTP Method:** `POST`
- **Summary:** Resolve company names to canonical ISINs and symbols
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `DiagnosisView.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade resolve company names to canonical isins and symbols. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `DiagnosisView.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/market-data/securities/resolve-text)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/market-data/securities/resolve-text" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`POST /api/v1/market-data/parse-financial-expression` — Parse Indian financial expressions (Crore, Lakh, %)

A. API Overview:

- **Endpoint:** `/api/v1/market-data/parse-financial-expression`
- **HTTP Method:** `POST`
- **Summary:** Parse Indian financial expressions (Crore, Lakh, %)
- **Router Module:** `backend/api/v1/market-data.py`
- **Frontend Caller:** `PortfolioGenerator.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade parse indian financial expressions (crore, lakh, %). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `PortfolioGenerator.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (POST /api/v1/market-data/parse-financial-expression)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
```

```
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `securities`, `market_prices`, `economic_indicators`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X POST "http://localhost:8000/api/v1/market-data/parse-financial-expression" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

8. Notifications & System Alerts

`GET /api/v1/notifications` — Retrieve in-app notifications for current user

A. API Overview:

- **Endpoint:** `/api/v1/notifications`
- **HTTP Method:** `GET`
- **Summary:** Retrieve in-app notifications for current user
- **Router Module:** `backend/api/v1/notifications.py`
- **Frontend Caller:** `NotificationsPanel.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve in-app notifications for current user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `NotificationsPanel.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/notifications)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `notifications`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/notifications" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/notifications/unread-count` — Retrieve count of unread notifications

A. API Overview:

- **Endpoint:** `/api/v1/notifications/unread-count`
- **HTTP Method:** `GET`
- **Summary:** Retrieve count of unread notifications
- **Router Module:** `backend/api/v1/notifications.py`
- **Frontend Caller:** `Navbar.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve count of unread notifications. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `Navbar.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/notifications/unread-count)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "unread_count": 2
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `notifications`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/notifications/unread-count" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`PUT /api/v1/notifications/{notification_id}/read` — Mark a single notification as read

A. API Overview:

- **Endpoint:** `/api/v1/notifications/{notification_id}/read`
- **HTTP Method:** `PUT`

- **Summary:** Mark a single notification as read
- **Router Module:** `backend/api/v1/notifications.py`
- **Frontend Caller:** `NotificationsPanel.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade mark a single notification as read. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `NotificationsPanel.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `notification_id` (UUID/String, required): Target entity identifier.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (PUT /api/v1/notifications/{notification_id}/read)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `notifications`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X PUT "http://localhost:8000/api/v1/notifications/n1/read" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`PUT /api/v1/notifications/mark-all-read` — Mark all notifications as read for current user

A. API Overview:

- **Endpoint:** `/api/v1/notifications/mark-all-read`
- **HTTP Method:** `PUT`
- **Summary:** Mark all notifications as read for current user
- **Router Module:** `backend/api/v1/notifications.py`
- **Frontend Caller:** `NotificationsPanel.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade mark all notifications as read for current user. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `NotificationsPanel.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:** None.
- **Request Body Content-Type:** `application/json`

```
{
  "action": "execute",
  "parameters": {}
}
```

F. Processing and Internal Workflow:

```
Client Request (PUT /api/v1/notifications/mark-all-read)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `notifications`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X PUT "http://localhost:8000/api/v1/notifications/mark-all-read" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`DELETE /api/v1/notifications/{notification_id}` — Dismiss and delete a notification

A. API Overview:

- **Endpoint:** `/api/v1/notifications/{notification_id}`
- **HTTP Method:** `DELETE`
- **Summary:** Dismiss and delete a notification
- **Router Module:** `backend/api/v1/notifications.py`
- **Frontend Caller:** `NotificationsPanel.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade dismiss and delete a notification. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `NotificationsPanel.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** Yes (`Authorization: Bearer <JWT_TOKEN>`)
- **Authorization:** Tenant isolation enforced (`user_id == current_user.user_id`).
- **Missing/Expired Token:** Returns `401 Unauthorized` with `detail: 'Could not validate credentials'`.

E. Request Contract:

- **Path Parameters:**
  - `notification_id` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (DELETE /api/v1/notifications/{notification_id})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
```

```
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: `notifications`.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X DELETE "http://localhost:8000/api/v1/notifications/n1" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

9. Link Verification & Disclosures

`GET /api/v1/links/verify` — Verify URL safety and reachability against allowed domains

A. API Overview:

- **Endpoint:** `/api/v1/links/verify`
- **HTTP Method:** `GET`
- **Summary:** Verify URL safety and reachability against allowed domains
- **Router Module:** `backend/api/v1/links.py`
- **Frontend Caller:** `lib/linkUtils.ts`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade verify url safety and reachability against allowed domains. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `lib/linkUtils.ts` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/links/verify)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/links/verify" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/links/filings/{symbol}` — Retrieve official BSE/NSE corporate announcement links

A. API Overview:

- **Endpoint:** `/api/v1/links/filings/{symbol}`
- **HTTP Method:** `GET`
- **Summary:** Retrieve official BSE/NSE corporate announcement links
- **Router Module:** `backend/api/v1/links.py`
- **Frontend Caller:** `StockWhyModal.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve official bse/nse corporate announcement links. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `StockWhyModal.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:**
  - `symbol` (UUID/String, required): Target entity identifier.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/links/filings/{symbol})
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/links/filings/TCS" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/links/regulatory` — Retrieve official SEBI, SCORES, and SMART ODR links

A. API Overview:

- **Endpoint:** `/api/v1/links/regulatory`
- **HTTP Method:** `GET`
- **Summary:** Retrieve official SEBI, SCORES, and SMART ODR links
- **Router Module:** `backend/api/v1/links.py`
- **Frontend Caller:** `Footer.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve official sebi, scores, and smart odr links. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `Footer.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/links/regulatory)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/links/regulatory" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/v1/links/audit-report` — Retrieve system link audit report

A. API Overview:

- **Endpoint:** `/api/v1/links/audit-report`
- **HTTP Method:** `GET`
- **Summary:** Retrieve system link audit report
- **Router Module:** `backend/api/v1/links.py`
- **Frontend Caller:** `Footer.tsx`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade retrieve system link audit report. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `Footer.tsx` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/v1/links/audit-report)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/v1/links/audit-report" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

10. System Health & Diagnostics

`GET /` — API root service welcome and version check

A. API Overview:

- **Endpoint:** `/`
- **HTTP Method:** `GET`
- **Summary:** API root service welcome and version check
- **Router Module:** `backend/api/v1/main.py`
- **Frontend Caller:** `System`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade api root service welcome and version check. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `System` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/health` — Comprehensive backend health and DB status

A. API Overview:

- **Endpoint:** `/api/health`
- **HTTP Method:** `GET`
- **Summary:** Comprehensive backend health and DB status
- **Router Module:** `backend/api/v1/main.py`
- **Frontend Caller:** `System`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade comprehensive backend health and db status. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `System` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/health)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`

- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/health" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /health` — Kubernetes/Docker liveness health check

A. API Overview:

- **Endpoint:** `/health`
- **HTTP Method:** `GET`
- **Summary:** Kubernetes/Docker liveness health check
- **Router Module:** `backend/api/v1/main.py`
- **Frontend Caller:** `System`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade kubernetes/docker liveness health check. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `System` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /health)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
```

```
      │
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      │
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/health" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/db/stats` — PostgreSQL database table count and active connections

A. API Overview:

- **Endpoint:** `/api/db/stats`
- **HTTP Method:** `GET`
- **Summary:** PostgreSQL database table count and active connections
- **Router Module:** `backend/api/v1/stats.py`
- **Frontend Caller:** `System`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade postgresql database table count and active connections. It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `System` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/db/stats)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/db/stats" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

`GET /api/securities` — List active securities (Legacy root endpoint)

A. API Overview:

- **Endpoint:** `/api/securities`
- **HTTP Method:** `GET`
- **Summary:** List active securities (Legacy root endpoint)
- **Router Module:** `backend/api/v1/main.py`
- **Frontend Caller:** `System`
- **Implementation Status:** Active & Verified in FastAPI

B. Why This API Is Used: This API is required by Lucid to provide institutional-grade list active securities (legacy root endpoint). It guarantees consistent transactional state, validates inputs against SEBI guidelines, and prevents data contamination between separate retail investor accounts.

C. When This API Is Called: Triggered by the user interacting with `System` or executed automatically by background services during portfolio lifecycle events.

D. Authentication & Authorization:

- **Authentication Required:** No (Public Endpoint)

E. Request Contract:

- **Path Parameters:** None.
- **Request Body:** None.

F. Processing and Internal Workflow:

```
Client Request (GET /api/securities)
      |
      ▼
FastAPI Security Header & Rate Limiter Middleware
      |
      ▼
JWT Verification (Security Bearer Dependency)
      |
      ▼
Pydantic Input Schema Validation
      |
      ▼
Domain Service Execution & PostgreSQL ACID Transaction
      |
      ▼
JSON Response Serialization to Client
```

G. Response Contract:

- **HTTP Success Status:** `200 OK`
- **Response Content-Type:** `application/json`

```
{
  "status": "success",
  "data": {}
}
```

H. Error Responses:

HTTP Status	Error Condition	Cause	Resolution
`400 Bad Request`	Malformed parameters	Invalid date or format	Verify parameter syntax
`401 Unauthorized`	Invalid JWT Token	Stale or missing Bearer token	Re-authenticate via `/api/v1/auth/login`
`403 Forbidden`	Multi-Tenant Breach	Accessing another user's resource	Verify resource ownership
`404 Not Found`	Entity Missing	Record does not exist in DB	Verify entity UUID
`422 Unprocessable`	Schema Validation	Missing required Pydantic field	Ensure all mandatory fields are provided
`429 Rate Limited`	Rate Limit Exceeded	More than allowed requests per min	Back off and retry

I. Database Interaction:

- Tables Read/Modified: Read-only metadata query.
- Transaction Safety: Wrapped in SQLAlchemy session context (`with SessionLocal() as db:`) with automatic rollback on error.

J. External Dependencies:

- External Dependencies: None (Internal Relational / Algorithmic Engine).

K. Example Usage (cURL):

```
curl -X GET "http://localhost:8000/api/securities" \
  -H "Content-Type: application/json"
```

L. Related APIs: `GET /api/health` → `GET /api/v1/market-data/health`

---

13. Frontend-to-Backend API Mapping

Every user interaction in the Next.js React 19 presentation layer maps directly to an implemented backend API:

Frontend Component	User Action / Trigger	HTTP Method	Backend API Endpoint	Request Data Sent	Consumed Response Data
`AuthView.tsx`	User submits login form	`POST`	`/api/v1/auth/login`	`email`, `password`	`access_token`, user profile
`AuthView.tsx`	User registers new account	`POST`	`/api/v1/auth/register`	`email`, `password`, `display_name`	User snapshot, login prompt
`Navbar.tsx`	User clicks logout	`POST`	`/api/v1/auth/logout`	`token` header	Revocation confirmation
`Navbar.tsx`	Polling badge counter (every 30s)	`GET`	`/api/v1/notifications/unread-count`	Bearer Token	`unread_count` integer
`RiskQuestionnaireView.tsx`	User opens risk profiler	`GET`	`/api/v1/risk-questions`	None	List of 8 psychometric questions
`RiskQuestionnaireView.tsx`	User finishes questionnaire	`POST`	`/api/v1/risk-assessments`	Answer question-option pairs	Risk score (0-100), risk category
`PortfolioGenerator.tsx`	User clicks 'Generate Portfolio'	`POST`	`/api/v1/portfolio-generation-requests`	Amount, horizon, risk, prompt	`request_id`, initial state
`PortfolioGenerator.tsx`	Polling generation status	`GET`	`/api/v1/portfolio-generation-requests/{id}`	Request ID	Multi-agent stage, final version
`HoldingsView.tsx`	User views active portfolio	`GET`	`/api/v1/portfolios/{id}/versions/{vid}`	Portfolio and Version UUID	Holdings, integer shares, weights
`ExplainabilityView.tsx`	User views research evidence	`GET`	`/api/v1/portfolios/{id}/versions/{vid}/explanations`	Version UUID	Agent findings, citations, traces
`StockWhyModal.tsx`	User clicks 'Why This Stock?'	`GET`	`/api/v1/portfolios/{id}/versions/{vid}/stocks/{sym}/why`	Stock Symbol	Level 1 thesis & Level 2 factors
`StockWhyModal.tsx`	User asks question about stock	`POST`	`/api/v1/portfolios/{id}/versions/{vid}/stocks/{sym}/chat`	User prompt text	Fiduciary AI response
`RebalancePreviewModal.tsx`	User clicks 'Preview Rebalance'	`GET`	`/api/v1/portfolios/{id}/rebalance-preview`	Portfolio UUID	Proposed trades, statutory taxes
`RebalancePreviewModal.tsx`	User clicks 'Confirm Rebalance'	`POST`	`/api/v1/portfolios/{id}/apply-rebalance`	Rebalance notes	Updated portfolio version
`DiagnosisView.tsx`	User uploads Demat CSV file	`POST`	`/api/v1/diagnosis/upload-csv`	CSV multipart stream	Parsed normalized holdings
`DiagnosisView.tsx`	User clicks 'Analyze Demat'	`POST`	`/api/v1/diagnosis/analyze/{id}`	Imported portfolio UUID	HHI score, concentration risks
`DiagnosisCopilotModal.tsx`	User queries imported holdings	`POST`	`/api/v1/diagnosis/{id}/chat`	User query prompt	AI copilot diagnosis commentary
`NotificationsPanel.tsx`	User opens alerts flyout	`GET`	`/api/v1/notifications`	Bearer Token	Array of unread/read alerts
`NotificationsPanel.tsx`	User clicks 'Mark All Read'	`PUT`	`/api/v1/notifications/mark-all-read`	Bearer Token	Success confirmation
`StressScenariosView.tsx`	User simulates RBI rate hike	`POST`	`/api/v1/portfolios/{id}/scenarios`	Scenario code (e.g. `RATE_HIKE`)	Simulated NAV drawdown
`BenchmarksView.tsx`	User compares vs Nifty 50	`GET`	`/api/v1/portfolios/benchmark-evaluation`	Portfolio UUID	Alpha, Beta, Sharpe comparison
`ProfileAuthModal.tsx`	User requests data export	`GET`	`/api/v1/profile/export`	Bearer Token	Complete GDPR/DPDP JSON archive

Frontend Component	User Action / Trigger	HTTP Method	Backend API Endpoint	Request Data Sent	Consumed Response Data
`ProfileAuthModal.tsx`	User permanently deletes account	`POST`	`/api/v1/profile/delete-account`	Password & confirmation phrase	Deletion status confirmation

---

14. API Workflow & Sequence Diagrams

All API sequence diagrams are rendered and stored in `docs/diagrams/api/`:

- `API_AUTH_FLOW.mmd` / `.svg` / `.png`
- `API_PORTFOLIO_GENERATION_FLOW.mmd` / `.svg` / `.png`
- `API_MONITORING_REBALANCE_FLOW.mmd` / `.svg` / `.png`
- `API_DIAGNOSIS_FLOW.mmd` / `.svg` / `.png`

---

15. Postman Suite & Verification Consistency

The API contracts documented in this specification are 100% synchronized with the Postman assets:

- **Collection:** `postman/Lucid_API_Collection.json`
- **Environment:** `postman/Lucid_Development_Environment.json`
- **Guide:** `postman/README.md`